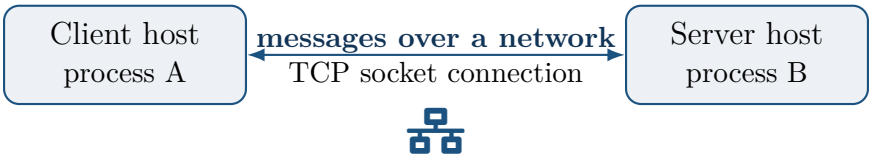
Vanier College

Computer Science Technology

420-511-VA Web Services

Week 1

Introduction to Distributed Systems and Interprocess Communication



Topic:	Distributed systems and interprocess communication
Scope:	Architectures, IPC, networking, TCP/UDP, sockets, framing, protocols, failures, and concurrency
Author:	Pravish Sainath
Term:	Fall 2026

A recap of concepts from several sources refined for clarity.

Contents

How to Use These Theory Notes 1

Week 1 Theory Learning Outcomes 1

1 The Big Picture: From a Local Program to a Distributed Service 2
1.1 Running example: opening a school portal . . . 2

2 What Is a Distributed System? 4
2.1 Examples and non-examples . . . 4
2.2 Distribution is not the same as parallelism . . . 4

3 Why Build Distributed Systems? Benefits and Costs 5
3.1 Common motivations . . . 5
3.2 The unavoidable costs . . . 5
3.3 Concrete systems: capabilities and costs together . . . 6

4 Essential Vocabulary 6

5 Fundamental Properties and Challenges 7
5.1 Concurrency . . . 8
5.2 No single perfectly shared clock . . . 8
5.3 Independent failures . . . 9
5.4 Transparency . . . 9

6 Common Distributed Architectures and Communication Patterns 10
6.1 Client-server . . . 10
6.2 Multi-tier architecture . . . 11
6.3 Peer-to-peer . . . 12
6.4 Request-response . . . 12
6.5 One-way messaging and work queues . . . 13
6.6 Publish-subscribe . . . 13
6.7 Streaming . . . 13
6.8 Load-balanced replicated services . . . 14
6.9 API gateway and aggregation . . . 14
6.10 Event-driven architecture . . . 14
6.11 Fan-out and fan-in . . . 15
6.12 One application can combine many patterns . . . 15
6.13 Pattern comparison and selection . . . 15

7 Interprocess Communication (IPC) 17
7.1 Why ordinary variables are not enough . . . 17
7.2 Two broad IPC models . . . 17
7.3 Choosing an IPC mechanism by situation . . . 18

8 Networking Essentials for Remote IPC 19
8.1 Hosts, names, and IP addresses . . . 19
8.2 Port numbers . . . 20
8.3 Endpoints and connections . . . 20
8.4 A simplified protocol stack . . . 21

8.5 A response-time mental model . . . 21

9 TCP and UDP: Two Transport Choices 22
9.1 TCP . . . 22
9.2 UDP . . . 22
9.3 Choosing between TCP and UDP in context . . . 23

10 The Socket Abstraction 24
10.1 The TCP client/server lifecycle . . . 25
10.2 Blocking operations . . . 25

11 TCP Is a Byte Stream: Messages Need Framing 26
11.1 Common framing strategies . . . 26
11.2 Character encoding . . . 27
11.3 Serialization . . . 27

12 Application Protocols: Giving Meaning to Bytes 28
12.1 A simple line-oriented calculator protocol . . . 28
12.2 Message grammar . . . 28
12.3 Validation rules . . . 29
12.4 Sequencing and protocol deadlock . . . 29

13 Sequential and Concurrent Servers 30
13.1 Sequential server . . . 30
13.2 Concurrent server . . . 30

14 Failures, Timeouts, and Defensive Design 31
14.1 Common failure symptoms . . . 31
14.2 Timeouts . . . 32
14.3 Defensive protocol rules . . . 32

15 Security Foundations for Raw Sockets 33

16 Worked Design: From Requirement to Protocol 33
16.1 Step 1: Identify roles and endpoint . . . 33
16.2 Step 2: Define requests and responses . . . 34
16.3 Step 3: Define sequencing . . . 34
16.4 Step 4: Anticipate failures . . . 34
16.5 Step 5: Separate responsibilities . . . 34

17 Theory Consolidation 35
17.1 Self-check questions . . . 35

A Suggested Answers to Theory Self-Check 36

B Glossary 37

References and Further Reading 39

How to Use These Theory Notes

A three-pass learning strategy

1. **First pass – build the mental model.** Read the definitions, diagrams, and contextual examples. Focus on autonomy, communication, state, roles, endpoints, and failure.
2. **Second pass – trace interactions.** For every architecture or sequence diagram, identify the client, server, state owner, message direction, possible blocking point, and possible partial failure.
3. **Third pass – explain and test yourself.** Use the self-check questions, glossary, and worked calculator design to explain each concept without relying on memorized wording.

These notes are more detailed than a slide deck and are designed for preparation, guided in-class use, and review. Repeated material is retained only where it serves a deliberate recap, comparison, checklist, or quick-reference purpose.

Important scope boundary

Week 1 explains the foundation beneath Web services: distributed execution, process-to-process communication, transport protocols, sockets, and simple application protocols. HTTP and REST are mentioned only as future examples; their detailed treatment belongs to later weeks.

Week 1 Theory Learning Outcomes

By the end of the theory component, a student should be able to:

1. explain what makes a system distributed and identify its benefits and unavoidable challenges;
2. distinguish a host, process, thread, client, server, service, endpoint, socket, connection, and protocol;
3. recognize common distributed architectures and communication patterns in everyday applications;
4. compare major IPC mechanisms and explain why sockets are suitable across hosts;
5. explain the roles of names, IP addresses, port numbers, TCP, UDP, and application-level message framing;
6. trace the complete conceptual lifecycle of a TCP client/server interaction;
7. design a small line-oriented application protocol with success and error responses;
8. identify blocking operations, partial failures, timeout needs, retry risks, and basic concurrency concerns.

1. The Big Picture: From a Local Program to a Distributed Service

A local program can keep all work in one process, share memory, and use ordinary method calls. A distributed application moves some responsibilities to independent processes, possibly on other hosts. Because those processes cannot directly share ordinary memory or method calls, they coordinate through an explicit communication mechanism.

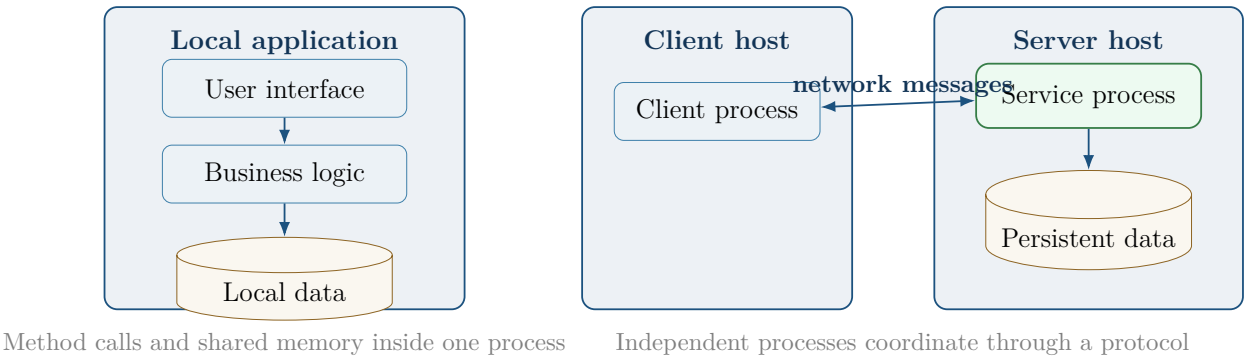


Figure 1: A local program keeps components in one process; a distributed application separates them and replaces some direct calls with message exchange.

The central transformation is therefore:

local method call ⟶ serialized message ⟶ network transfer ⟶ remote processing

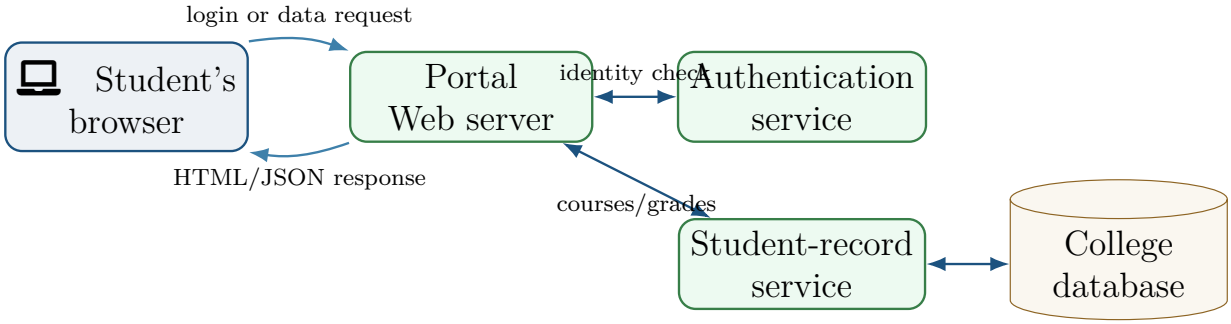
This replacement changes the programming model: communication, representation, timing, and failure behavior must now be designed explicitly. Later sections examine the resulting benefits and engineering costs.

Web-service preview

Later in the course, an HTTP client may send a request such as `GET /students/42` to a RESTful service. Week 1 looks below that convenient HTTP abstraction. At the transport level, two processes still need endpoints, a connection or datagram exchange, bytes, framing rules, and failure handling.

1.1 Running example: opening a school portal

A student experiences the college portal as one application, although a single click may cross several autonomous components. The following illustration applies the general transformation from Figure 1.



Conceptual illustration. One portal action can cross several process, service, and data boundaries even though the student sees one page.

Component	Primary responsibility	Distributed-system implication
Browser	Present the interface, collect input, and initiate requests.	It is a client of the portal and must handle delays, unavailable service, and stale views.
Portal Web server	Route requests, manage the user session, validate basic input, and coordinate backend calls.	It is a server to the browser and a client to authentication and student-record services.
Authentication service	Verify identity and issue or validate session credentials.	Its failure can prevent otherwise healthy portal functions from completing.
Student-record service	Apply academic rules and expose courses, grades, schedules, or registration operations.	Concurrent requests must preserve rules such as prerequisites and seat limits.
Database	Persist students, courses, registrations, and audit records.	It is shared state and therefore a major consistency, performance, and availability dependency.

Key Points to Remember

The user sees one portal, but the implementation contains several independently executing components. Every boundary replaces a direct in-process operation with communication, delay, validation, and possible partial failure.

2. What Is a Distributed System?

Distributed system

A **distributed system** is a collection of autonomous computing elements that coordinate through communication and are intended to provide a coherent service or experience. The elements may be processes, machines, containers, mobile devices, sensors, or cloud instances.

Three ideas are essential:

1. **Autonomy:** each element has its own execution state and can continue, pause, fail, or restart independently.
2. **Communication:** elements coordinate by exchanging information, usually over a network.
3. **Coherence:** users normally care about the service as a whole rather than the internal location of every component.

2.1 Examples and non-examples

System	Distributed elements	Why it is distributed
Online store	Browser, API server, payment provider, inventory service, database	Independent processes exchange requests and responses across network boundaries.
Cloud file storage	Client devices, metadata service, object-storage nodes, replication system	Data and responsibility are spread over several nodes while appearing as one service.
Multiplayer game	Player clients, game servers, matchmaking service	State changes originate on many devices and must be coordinated across a network.
Single Java process	One process and one address space	Threads may be concurrent, but all execution remains inside one process.
One computer running three separate processes	Three independent processes on one host	It is distributed at the process level even though no physical network separates the processes.

2.2 Distribution is not the same as parallelism

Parallelism means performing multiple computations at the same time, often to finish a computation faster. **Distribution** means coordinating autonomous elements through communication. A system can be:

- parallel but not distributed, such as several CPU cores working inside one process;
- distributed but not highly parallel, such as a client waiting for one remote server response;

- both parallel and distributed, such as a large search engine using many servers concurrently.

⚠ The most important mindset change

A remote call is not simply a slower local call. The caller may send a request and then lose contact. It may be impossible to know immediately whether the server never received the request, received it but failed, completed it but lost the response, or is merely slow. Every distributed program must make this uncertainty explicit.

3. Why Build Distributed Systems? Benefits and Costs

3.1 Common motivations

Goal	Meaning	Example
Resource sharing	Make data, hardware, or capabilities available beyond one process or host.	Many applications use one database or authentication service.
Scalability	Add resources or divide work as demand grows.	Several server instances handle more clients.
Availability	Continue serving users when some components fail.	Replicated servers provide failover.
Geographic reach	Place services or data closer to users.	A mobile application communicates with a remote cloud service.
Specialization	Let components use different technologies and evolve independently.	A payment service, recommendation service, and Web front end use different stacks.
Collaboration	Coordinate multiple users or devices around shared state.	A shared project-management or document-editing application.

3.2 The unavoidable costs

Distribution is not free. Replacing a local call with a remote interaction adds:

- **latency:** a remote operation must cross software and network boundaries;
- **partial failure:** one component or path can fail while others continue;
- **concurrency:** many clients may modify or observe state at overlapping times;
- **uncertain ordering:** message delay varies, and events are not automatically observed in the same order everywhere;
- **security exposure:** communication crosses trust boundaries and accepts remote input;
- **operational complexity:** deployment, monitoring, configuration, and version compatibility become part of the design.

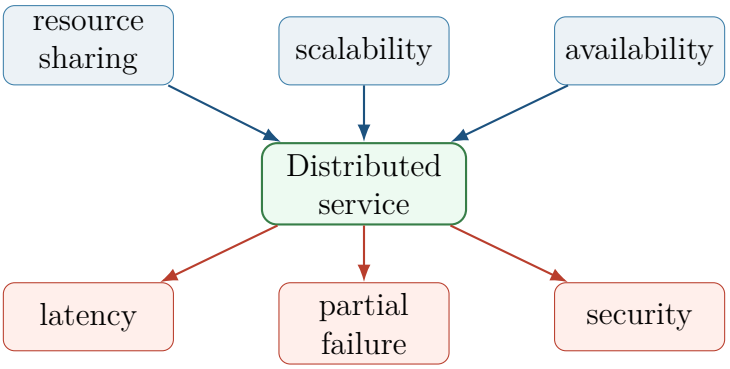


Figure 2: Distribution is chosen for useful capabilities, but every benefit arrives with engineering costs.

3.3 Concrete systems: capabilities and costs together

Familiar applications combine these motivations in different ways; their component boundaries determine which costs dominate.

Application	Typical distributed components	Benefits and design pressures
Instagram-like social application	Mobile clients, gateway, profile/feed services, media storage, notification and analytics workers, and replicated databases.	Enables global reach and scale; must tolerate temporarily stale feeds, distribute media efficiently, and isolate dependency failures.
School portal and course registration	Browser, portal, authentication, registration and timetable services, plus the student/course database.	Centralizes access, but overlapping requests compete for limited seats; correctness, auditability, and clear failure reporting are essential.
Concert or airline ticketing	Search, inventory, pricing, payment, order, and notification services.	Supports specialized scaling, but timeouts can leave seat holds or payments uncertain; duplicate-safe commands and explicit status checks are essential.

Design lesson

Do not select distribution only because it appears modern. Identify the capability that requires it, then identify the new latency, consistency, failure, security, and operational obligations created by each component boundary.

4. Essential Vocabulary

Term	Meaning
Host or node	A computing environment participating in the system. A physical machine can host several virtual machines or containers; one host can also run several processes.
Process	A running program with its own virtual address space and operating-system resources. Independent processes do not normally share ordinary variables.
Thread	A sequence of execution inside a process. Threads in one process share memory and must coordinate access to mutable state.
Client	The role that initiates an interaction by sending a request or opening a connection. A process can act as a client in one interaction and as a server in another.
Server	The role that waits for and handles client requests. A server process commonly listens at a known endpoint.
Service	A capability offered through an interface or protocol. Several server instances may jointly provide one logical service.
Request	A message asking another process to perform an operation or provide information.
Response	A message that communicates a result, status, or error after a request.
Endpoint	A communication address at which a process can be reached. For TCP and UDP, an endpoint is commonly represented by an IP address and port number.
Protocol	Agreed syntax, semantics, sequencing, and failure rules that allow independent implementations to communicate.
Socket	A programming abstraction representing a communication endpoint.
Connection	For TCP, a logical bidirectional byte-stream relationship between two endpoints. UDP is connectionless at the transport-service level.
Serialization	Conversion of structured data into bytes or text that can be transmitted and reconstructed.

Roles are contextual

A process is not permanently “a client” or “a server.” The browser is the client and an API is the server during an HTTP request. That same API may become a client when it calls a database proxy, authentication service, or third-party API.

5. Fundamental Properties and Challenges

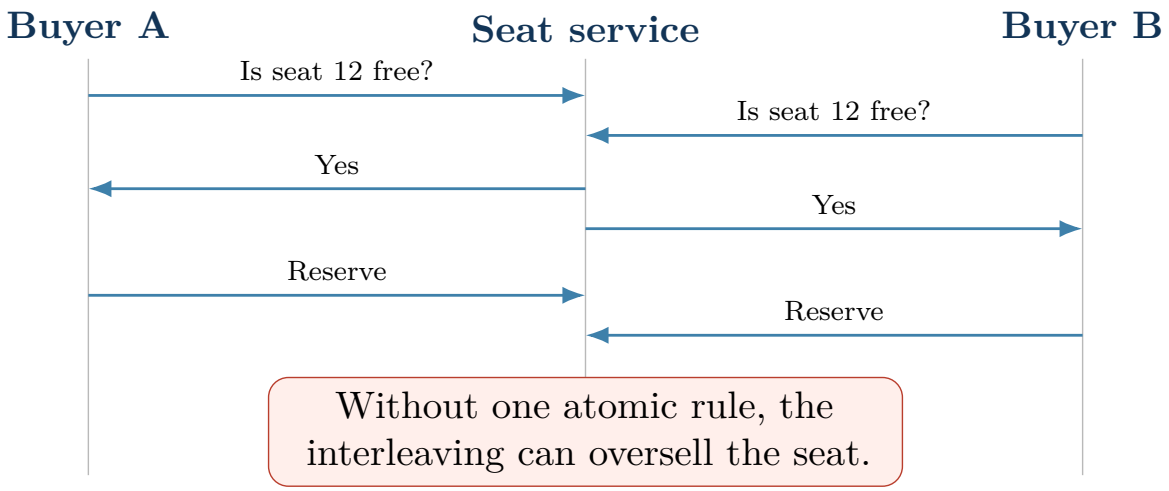
The following four characteristics explain many of the challenges introduced above.

5.1 Concurrency

Independent processes can issue overlapping requests at unpredictable times. Any server that owns shared mutable state must prevent unsafe interleavings.

Concurrency in ticketing: two buyers, one remaining seat

Buyer A and Buyer B can both ask whether seat 12 is free before either reservation is committed. If the service performs "check" and "reserve" as unrelated steps, both requests may observe "free" and both may appear to succeed. The system needs an atomic database operation, transaction, lock, version check, or another rule that makes the conflicting update safe.



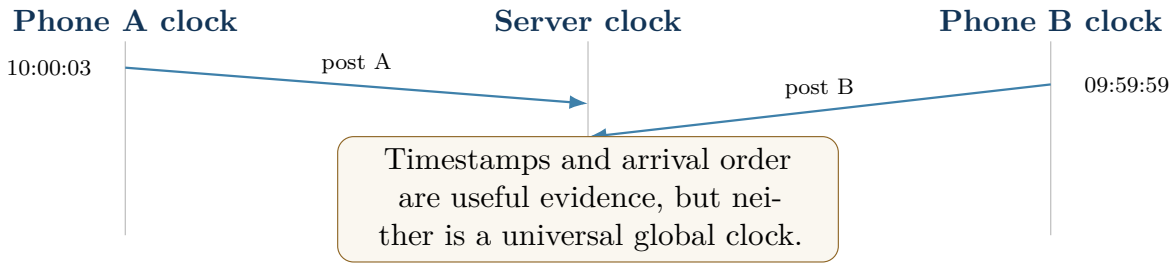
Conceptual illustration. Concurrency bugs arise from the interleaving of individually reasonable operations.

5.2 No single perfectly shared clock

Each host has a drifting local clock, while message delay varies. Timestamps help, but they do not create a perfect global event order.

Why timestamps alone may mislead

Suppose two phones create social-media posts. Phone A reports `10:00:03`; Phone B, whose clock is four seconds slow, reports `09:59:59`. Variable network delay may also cause post B to arrive first. Device time, arrival order, and the true order of user actions can therefore disagree. Systems often rely on server-assigned versions, sequence numbers, or application-specific ordering rules rather than assuming every clock is perfect.



Conceptual illustration. Clock drift and variable message delay complicate global event ordering.

5.3 Independent failures

A **partial failure** occurs when one component or communication path fails while other parts of the system remain healthy. The caller may observe only a missing or late response.

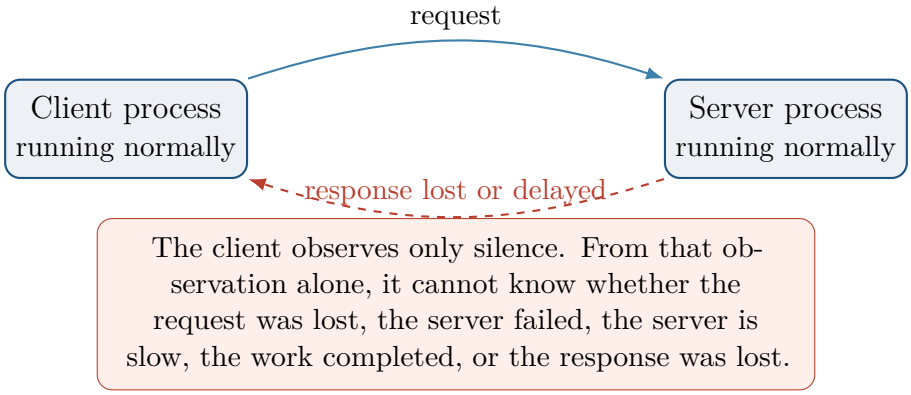


Figure 3: A partial-failure ambiguity: the request and response paths are distinct, and either direction can fail independently.

Four different histories can produce the same timeout

1. The request never reached the server.
2. The server received the request and failed before completing it.
3. The server completed the operation, but the response was lost.
4. The server and network were healthy but slower than the caller's deadline.

The client initially observes the same fact in all four cases: no usable response arrived in time. A retry may be necessary in case 1, harmless in case 2, dangerous in case 3, and wasteful in case 4. Safe retry behavior must therefore be part of the application design.

5.4 Transparency

A distributed system may hide selected implementation details while preserving important failure semantics. Common forms include:

- **access transparency:** similar operations are used for local and remote resources;

- **location transparency:** users do not need to know the physical host;
- **replication transparency:** several copies appear as one service;
- **failure transparency:** the system masks some failures through recovery or redundancy.

Transparency is selective: an API may hide socket and location details, but it must still report meaningful timeouts and unavailable dependencies.

Transparency	Everyday example	What must remain visible
Access	A portal displays local cached data and remote records through a similar interface.	The application must still distinguish unavailable or unauthorized remote data when the distinction matters.
Location	Students use one portal name without knowing the current server's IP address or data-centre location.	Operators still need endpoint, routing, and regional information for diagnosis.
Replication	Several portal instances appear as one service behind a load balancer.	Consistency delay, failover, and stale replicas may still influence behavior.
Failure	A retry or replica hides a brief component failure.	Persistent failure, uncertain completion, and user action requirements must be reported honestly.

6. Common Distributed Architectures and Communication Patterns

Architecture and communication pattern are related but different

An **architecture** organizes responsibilities: where presentation, business logic, state, coordination, and trust boundaries reside. A **communication pattern** organizes interaction: who initiates, whether a reply is immediate, whether a broker is involved, how many consumers receive data, and whether the exchange is short-lived or continuous. Real applications normally combine several architectures and patterns.

6.1 Client-server

The client-server pattern assigns complementary roles: a client initiates a request or connection, while a server listens for and handles the interaction.

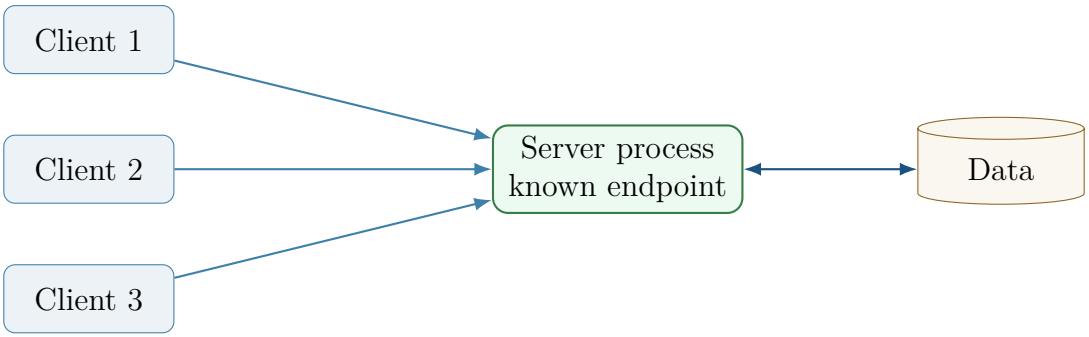
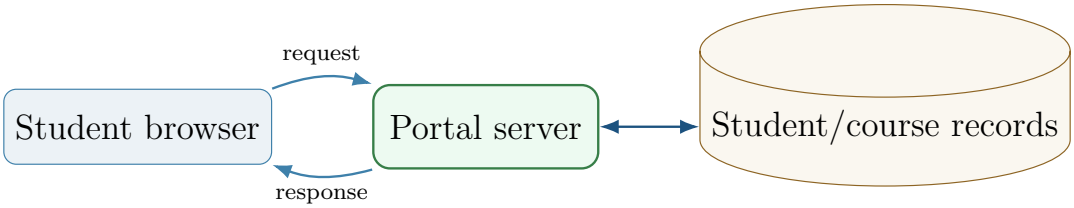


Figure 4: In client-server architecture, clients initiate interactions with a service at a known endpoint.

Roles are contextual. The portal serves the browser but becomes a client when it calls authentication or student-record services. Other everyday pairs include a mobile banking application and its API, or a game client and game server.



School-portal mapping. The browser presents information; the server validates rules; the database persists shared state. Business rules such as prerequisites belong on the trusted server side, not only in the browser.

6.2 Multi-tier architecture

A multi-tier design separates presentation, application logic, and data storage so that each tier can evolve or scale independently.

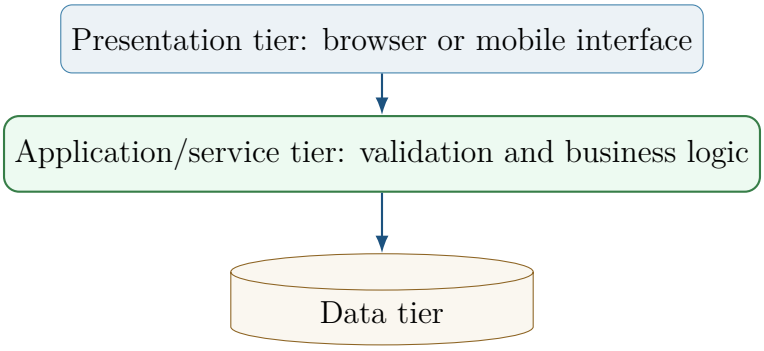


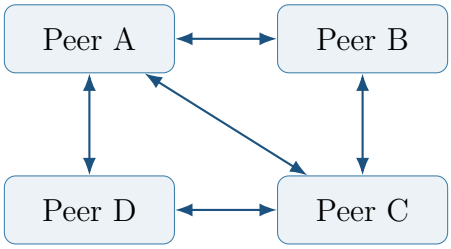
Figure 5: A simple three-tier architecture separates presentation, service logic, and persistent data.

Ticketing system mapped to three tiers

The browser displays a seat map and collects a choice. The application tier enforces holds, pricing, eligibility, and payment workflow. The data tier persists seat inventory, orders, payment references, and audit records. The browser may improve usability, but the application tier must make the authoritative decision because client-side code can be stale or modified.

6.3 Peer-to-peer

In a peer-to-peer design, each participant can request and provide capabilities. Pure designs have no permanent central server, although practical systems often retain central discovery, identity, or coordination.

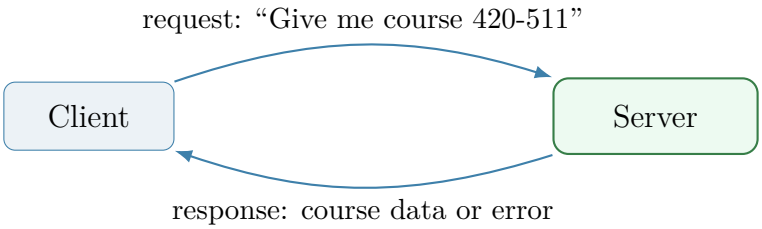


Conceptual illustration. Each peer can both request and provide capabilities. Peers may join, leave, or fail independently.

Examples include BitTorrent, some blockchain networks, and direct device collaboration. Key questions concern discovery, identity, trust, changing membership, and data availability when peers leave.

6.4 Request-response

A client sends one request and expects one matching response or error, as in HTTP APIs, database calls, and the Week 1 calculator protocol.

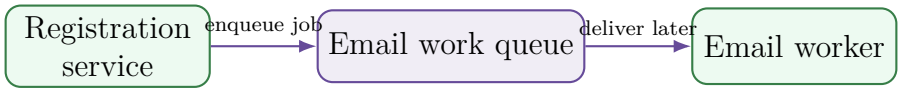


Request-response. The caller waits or awaits asynchronously and therefore needs timeout, error, and cancellation semantics.

Because the caller expects a timely reply, request-response creates **temporal coupling** and needs timeout, error, and cancellation rules. A timeout still does not reveal whether the remote operation occurred.

6.5 One-way messaging and work queues

A sender emits a job without waiting for final completion, and a queue buffers it until a consumer is ready.



Work-queue example. Registration can finish without making the student wait for an email server.

A work queue reduces temporal coupling, absorbs demand spikes, and lets workers scale independently. At the job level it is commonly point-to-point: one worker claims a queued item. The design must still specify delivery guarantees, retry limits, ordering, duplicate handling, and a policy for repeatedly failing messages.

6.6 Publish-subscribe

Producers publish events to a topic or broker. Interested consumers subscribe without the producer calling each one directly.

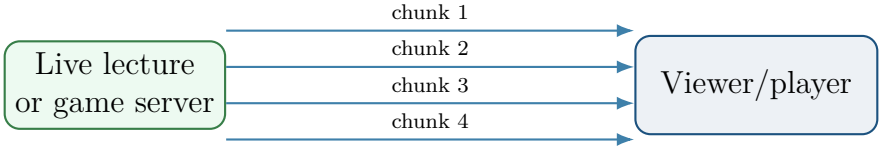


Publish-subscribe. One event fans out to several consumers, and new subscribers can be added without changing the producer.

Publish-subscribe is the one-to-many communication pattern: one publication can reach several subscribers without the producer calling them individually. Subscribers must handle event-version changes, repeated delivery, late arrival, and independent processing times.

6.7 Streaming

A streaming interaction carries a sequence of values or media chunks over time rather than one isolated response.

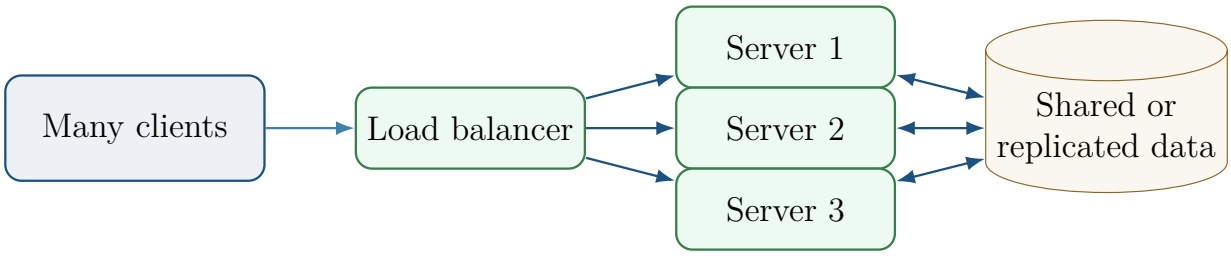


Streaming. A continuing sequence requires buffering, flow control, termination, reconnection, and recovery rules.

Examples include media playback, telemetry, live location, game updates, and server-sent events. The protocol must define acceptable delay and whether missing old data is retransmitted or skipped.

6.8 Load-balanced replicated services

Several server instances provide one logical service, and a load balancer selects an instance for each request or connection.

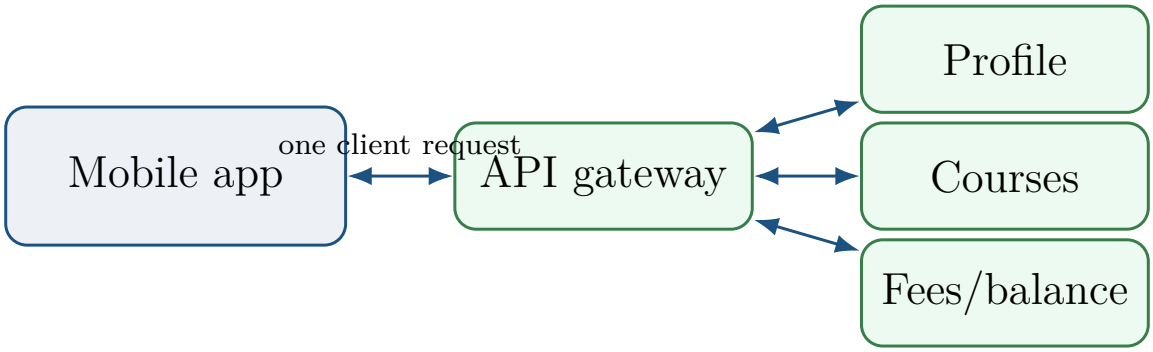


Load-balanced replicas. Clients perceive one service while traffic is spread across instances.

This pattern improves scale and can improve availability, while introducing decisions about health checks, overload, state location, session affinity, replication, and safe retry after instance failure.

6.9 API gateway and aggregation

A gateway provides one client entry point for routing, authentication, translation, or aggregation across internal services.

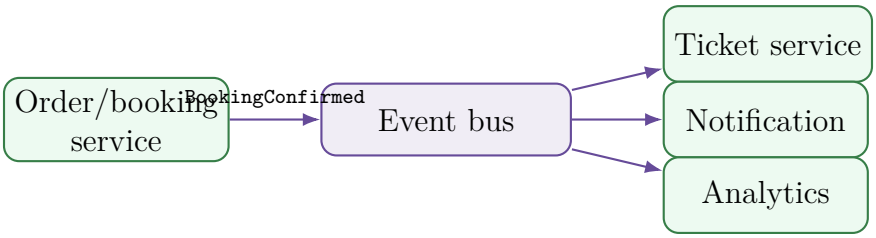


School-portal home screen. The gateway aggregates profile, courses, and balance into one response.

The client becomes simpler, while the gateway inherits downstream timeouts, partial-result policy, error translation, security enforcement, and bottleneck or single-point-of-failure risk.

6.10 Event-driven architecture

Components react to events rather than relying on one coordinator to call every step synchronously.

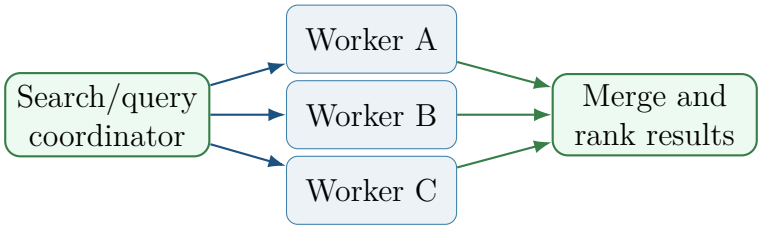


Event-driven workflow. Independent consumers react when a booking is confirmed.

Event-driven architecture applies event exchange to coordinate a larger workflow. Instead of synchronously calling every next step, a component records a state change and independent consumers react. This improves decoupling but requires event contracts, observability, eventual-consistency expectations, and duplicate-safe consumers.

6.11 Fan-out and fan-in

A coordinator divides work among several workers and combines their results.

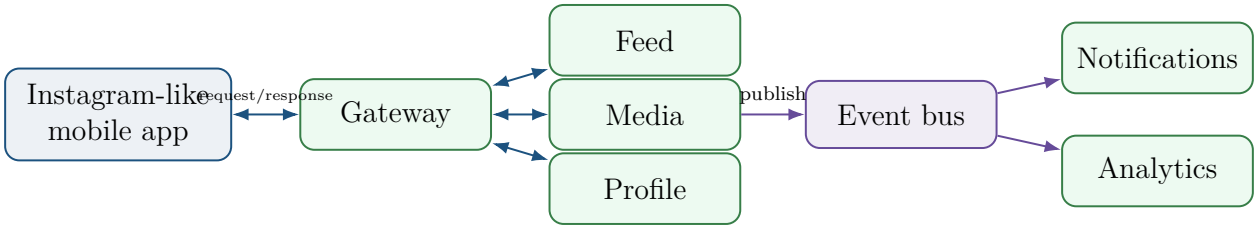


Fan-out/fan-in. Work is parallelized, but the coordinator must define deadlines and partial-result behavior.

Examples include searching several indexes, querying multiple travel providers, and parallel image processing. The coordinator needs a deadline so one failed or unusually slow worker cannot delay the entire result indefinitely.

6.12 One application can combine many patterns

An Instagram-like application can combine all of these patterns: client-server access, gateway aggregation, replicas, publish-subscribe, work queues, and live streaming.



Combined architecture. A single user action can cross synchronous and asynchronous patterns.

6.13 Pattern comparison and selection

Pattern	Basic idea	Typical implication
Request-response	A request is followed by a corresponding response.	Natural for commands and queries; the caller waits or awaits and needs timeout and error semantics.
One-way messaging/work queue	A sender emits work without waiting for final completion.	Reduces temporal coupling and buffers spikes; delivery, ordering, retry, and duplicate policies matter.
Publish-subscribe	Producers publish events; consumers subscribe by topic.	Supports one-to-many loose coupling; event versions, duplicates, and delayed processing matter.
Streaming	A long-lived interaction carries values over time.	Requires buffering, flow control, termination, reconnection, and recovery rules.
Load-balanced replicas	One logical endpoint routes to several server instances.	Improves scale and availability; health, state, and retry behavior must be designed.
Gateway/ aggregation	One component fronts or combines several services.	Simplifies clients; the gateway must translate downstream latency, errors, and partial failure.
Event-driven workflow	Components react to published state changes.	Supports asynchronous specialization; eventual consistency and observability become important.
Fan-out/fan-in	Split work among several workers and combine results.	Can reduce latency or broaden search; deadlines and missing-result rules are necessary.

Key Points to Remember

- Identify the components and their responsibilities before naming the pattern.
- Ask who initiates communication, who owns state, which component can fail independently, and whether the sender requires an immediate result.
- No pattern removes failure; each pattern moves or reshapes failure and coordination responsibilities.

Explore further: Microsoft Azure Architecture Center – Cloud Design Patterns provides concise descriptions of gateway aggregation, publisher-subscriber, queue-based load leveling, circuit breaker, and other common distributed patterns.

7. Interprocess Communication (IPC)

Interprocess communication

Interprocess communication (IPC) is the set of operating-system and networking mechanisms that allow independent processes to exchange data and coordinate their actions.

Because processes have separate virtual address spaces, ordinary references and variables do not cross the boundary. IPC must explicitly share controlled state or transfer encoded data.

7.1 Why ordinary variables are not enough

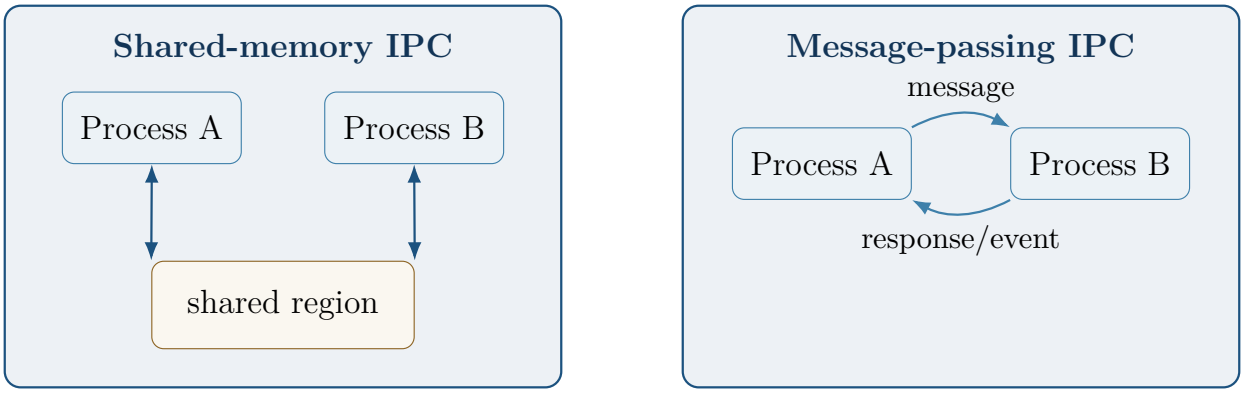
Suppose process A contains:

A variable inside one process

```
int count = 5;
```

Process B cannot read `count` by name because that name and address exist only inside process A. IPC must transfer the value, expose a shared region, or provide an operation.

7.2 Two broad IPC models



Processes coordinate access with locks or other synchronization

Data crosses a process boundary as explicitly encoded messages

Figure 6: Shared memory exposes a common region; message passing exchanges explicit units of information.

Mechanism	Basic idea	Strength	Main caution
Shared memory	Processes map a common memory region.	Very fast for large local data.	Synchronization and race conditions.

Mechanism	Basic idea	Strength	Main caution
Pipe or FIFO	A byte stream connects process output to process input.	Simple for related or local processes.	Usually local; framing is still needed.
Message queue	The operating system or broker stores messages.	Decoupling and buffering.	Delivery, ordering, and duplicate policies.
Signals/events	A process receives a lightweight notification.	Efficient control notification.	Very little payload and limited semantics.
Local-domain socket	Socket-like communication within one host.	Familiar API without IP routing.	Platform and deployment differences.
TCP socket	Connected reliable byte stream between endpoints.	Ordered, reliable, full-duplex communication.	No message boundaries; connection state and timeouts.
UDP socket	Independent datagrams between endpoints.	Low overhead and preserved datagram boundaries.	No guarantee of delivery, order, or duplicate suppression.

Key Points to Remember

- Shared memory communicates by sharing state; message passing communicates by transferring information.
- Sockets are especially important because the same abstraction can connect processes on one host or across a network.
- An IPC mechanism transports data; the application protocol defines what the data means.

7.3 Choosing an IPC mechanism by situation

Situation	Likely mechanism	Reasoning
Two functions inside one Java process	Ordinary method call or shared objects	No process boundary exists, so IPC would add unnecessary complexity.
Two local processes exchanging a large image buffer	Shared memory with synchronization	Avoids repeatedly copying a large payload, but access must be coordinated.

Situation	Likely mechanism	Reasoning
A shell command feeding another command	Pipe	Natural local byte-stream composition between producer and consumer.
A registration service handing email work to another process	Message queue	The sender can finish while work is buffered and retried independently.
A Java client calling a service on another host	TCP socket	Cross-host, ordered, reliable byte-stream communication fits request-response.
A sensor sending frequent measurements where an occasional old sample may be skipped	UDP datagrams	Preserved datagram units and low overhead may be more useful than retransmitting stale data.

Mechanism versus protocol

Choosing TCP, a queue, or shared memory selects how information moves. The application must still define the representation, meaning, legal sequence, validation, completion, and failure behavior of that information.

8. Networking Essentials for Remote IPC

8.1 Hosts, names, and IP addresses

A **host name**, such as `localhost` or `example.org`, is human-friendly. An **IP address**, such as IPv4 loopback `127.0.0.1` or IPv6 loopback `::1`, is used for packet delivery.

Name resolution maps a host name to one or more IP addresses; Java represents and resolves them with `InetAddress`.

From a portal name to a process

A browser may start with a name such as `portal.example.edu`. Name resolution returns an IP address. The client then connects to a transport port, commonly port 443 for HTTPS. The operating system delivers the traffic to the process bound to that endpoint. The name helps humans and permits deployment changes; the IP address routes packets; the port selects the transport endpoint.

Why use loopback in the first lab?

The loopback address routes traffic back to the same host. Running both client and server on `127.0.0.1` removes classroom-network and firewall uncertainty while preserving the socket programming model. Once the program works locally, it can be tested across machines with explicit authorization.

8.2 Port numbers

An IP address identifies a host or interface, not the receiving process. TCP and UDP use a 16-bit **port number** to distinguish transport endpoints.

$$0 \leq \text{port} \leq 65535$$

The conventional IANA ranges are:

Range	Numbers	Typical use
System ports	0–1023	Well-known services and privileged/system assignments.
User ports	1024–49151	Registered or application-selected services.
Dynamic/private ports	49152–65535	Temporary client ports and private use.

For a classroom server, choose an unused user-range port such as `5050`, `5500`, or `8088`; the operating system normally assigns the client's temporary local port.

8.3 Endpoints and connections

For introductory TCP and UDP programming, an endpoint can be written as:

$$\text{endpoint} = (\text{IP address},\ \text{port number})$$

A TCP connection is identified by both endpoints:

$$(\text{client IP}, \text{client port}, \text{server IP}, \text{server port})$$

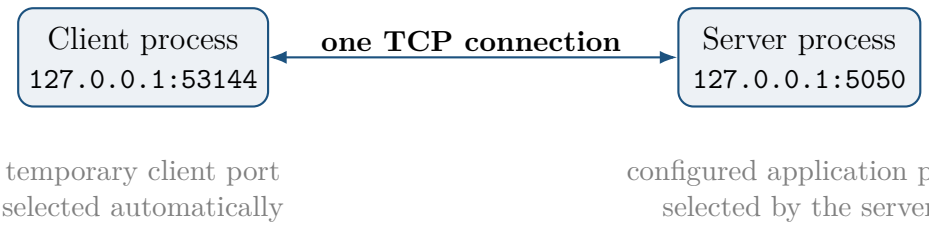


Figure 7: A TCP connection joins two endpoints. The client's local port is commonly allocated automatically.

Java's `InetSocketAddress` combines an IP address or host name with a port.

8.4 A simplified protocol stack

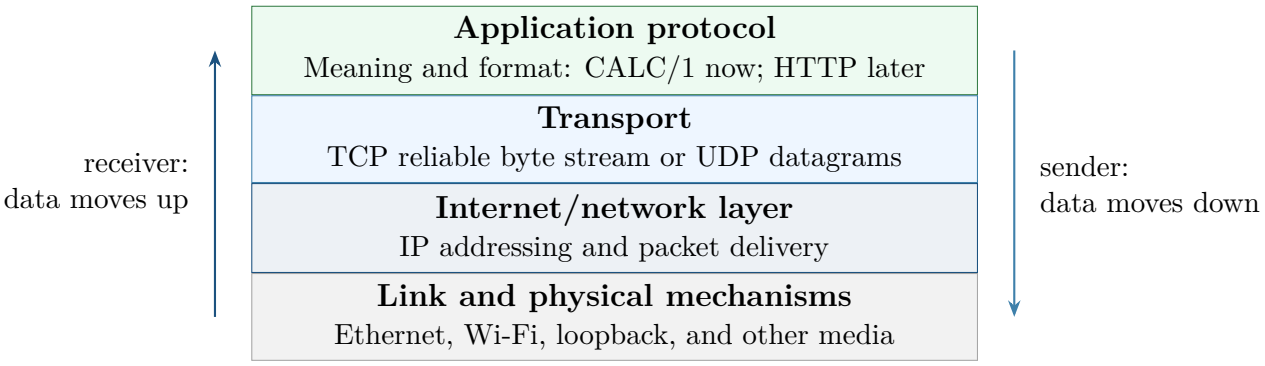


Figure 8: Applications use transport services, which use IP and lower-level network mechanisms.

Following `ADD 2 3` through the stack

1. The application protocol creates the UTF-8 line `ADD 2 3\n` and defines what that line means.
2. TCP treats the encoded characters as an ordered stream of bytes and manages connection, retransmission, flow, and congestion behavior.
3. IP places transport data into packets addressed to the destination host.
4. The link layer carries packets over loopback, Ethernet, Wi-Fi, or another local medium.
5. On the receiver, the path is reversed: link to IP to TCP to the server's application parser.

Each layer provides a service to the layer above without needing to understand the calculator operation itself.

8.5 A response-time mental model

A remote response time contains several components:

$$T_{\text{response}} = T_{\text{queue}} + T_{\text{request network}} + T_{\text{server work}} + T_{\text{response network}} + T_{\text{client processing}}$$

Network and queueing delay can dominate even fast server work, so avoid unnecessary dependent remote calls inside tight loops.

A numerical response-time example

Suppose queueing takes 20 ms, request delivery 15 ms, server work 30 ms, response delivery 15 ms, and client processing 5 ms. Then

$$T_{\text{response}} = 20 + 15 + 30 + 15 + 5 = 85 \text{ ms}.$$

Twenty dependent calls made one after another could therefore consume about 1.7 seconds even though the server computation totals only 600 ms. Batching, caching, parallel requests, or redesigning the interface may reduce the number of network round trips.

9. TCP and UDP: Two Transport Choices

9.1 TCP

The Transmission Control Protocol provides applications with a **reliable, in-order, bidirectional byte stream**. The current standards-track specification is RFC 9293. Important implications are:

- **Connection-oriented:** endpoints establish connection state before application data exchange.
- **Reliable delivery within the connection:** TCP detects loss and corruption and uses acknowledgments and retransmission.
- **Ordered bytes:** the receiver reads bytes in the same order in which the sender's stream accepted them.
- **Full duplex:** both endpoints can send and receive.
- **No application message boundaries:** TCP delivers a stream of bytes, not the original calls to `write()` or `println()`.
- **Flow and congestion control:** TCP regulates transmission according to receiver and network conditions.

⚠ What TCP reliability does not mean

A successful local write does not prove that the remote application processed the data. TCP can retransmit lost segments and preserve byte order, but the server may crash after receiving the request, the application may reject it, or the response may never return. End-to-end application correctness still requires protocol design, validation, acknowledgments, timeouts, and sometimes safe retry rules.

9.2 UDP

The User Datagram Protocol, specified by RFC 768, sends independent datagrams. UDP:

- preserves datagram boundaries;
- does not establish a TCP-style connection before sending;
- does not guarantee delivery, ordering, or duplicate suppression;
- has lower transport overhead and lets the application choose its own reliability behavior.

UDP suits cases where timeliness matters more than perfect delivery or the application can repair loss. Week 1 focuses on TCP; UDP is included for contrast.

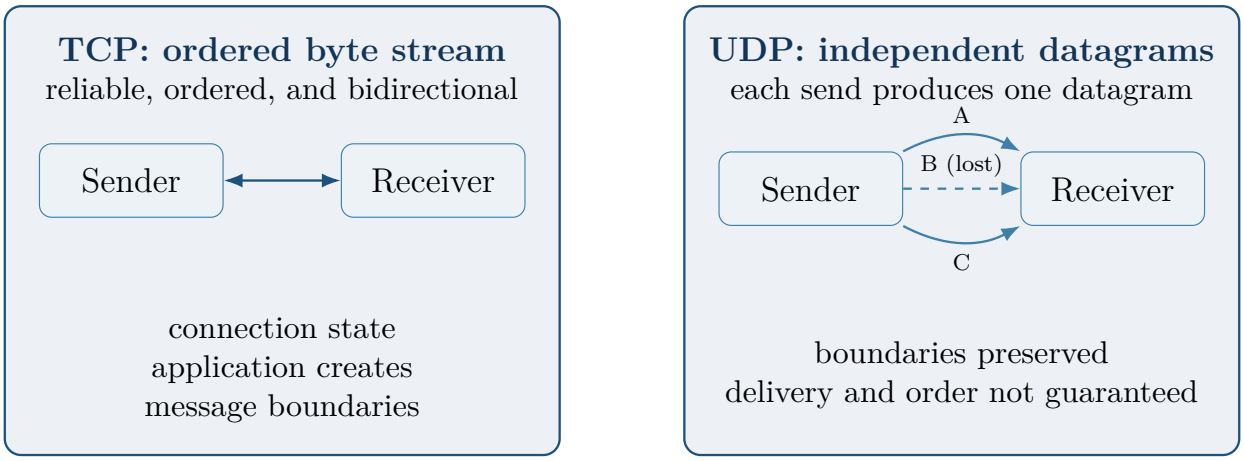


Figure 9: A high-level comparison of TCP and UDP.

Question	TCP	UDP
Connection state?	Yes, before application exchange.	No TCP-style connection setup.
Delivery guarantee?	Reliable within an established connection, unless the connection fails.	No transport-level delivery guarantee.
Ordering?	In-order byte stream.	Datagrams can arrive out of order.
Message boundaries?	No; the application creates framing.	Yes; one receive obtains one datagram up to the buffer limit.
Typical Week 1 use	Text request-response protocol.	Optional contrast exercise.

9.3 Choosing between TCP and UDP in context

Scenario	Likely choice	Reasoning
Course registration or payment command	TCP	Every request and response must be received in order, and application-level confirmation is required.
File upload or database backup	TCP	Missing or reordered bytes would corrupt the object; reliable ordered transfer is central.
Live game position updates	Often UDP or a UDP-based protocol	A new position may make an older lost update irrelevant; low delay can be more valuable than retransmitting stale data.

Scenario	Likely choice	Reasoning
Interactive voice/video	Often UDP-based real-time protocols	Small losses can be concealed, while delayed retransmissions may be noticeable. Modern systems may also use QUIC or other transports above UDP.
Simple name-resolution query	Commonly UDP, with protocol-specific retry or fallback	The request and response are small datagrams; the application protocol defines retry and alternate behavior.

Decision questions

Does every byte need to arrive? Must it arrive in order? Is old data still useful after delay? Can the application repair loss? Does the interaction need one continuing connection? Transport choice follows the application's correctness and timeliness requirements.

10. The Socket Abstraction

Socket

A **socket** is a programming abstraction for a communication endpoint. It gives an application methods to connect or bind, read or receive, write or send, inspect endpoint information, configure options, and close resources.

In the standard blocking Java API, two classes play different server-side roles:

- `ServerSocket` represents a listening TCP endpoint. It binds to a local port and accepts incoming connections.
- `Socket` represents one connected TCP endpoint. Both clients and accepted server connections use `Socket`.

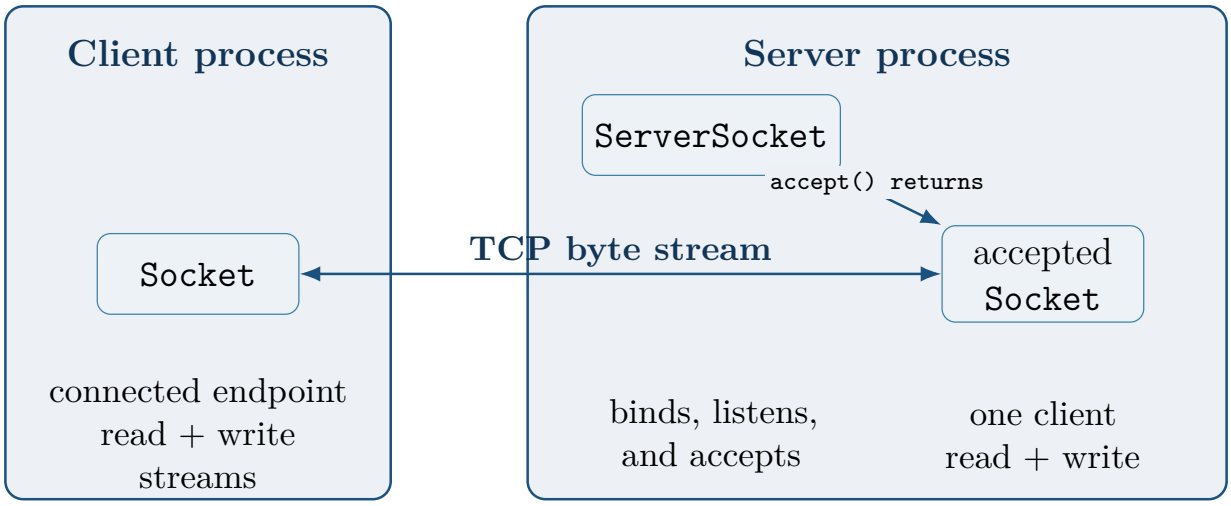


Figure 10: The server keeps a listening `ServerSocket`; each accepted connection is represented by a separate connected `Socket`.

10.1 The TCP client/server lifecycle

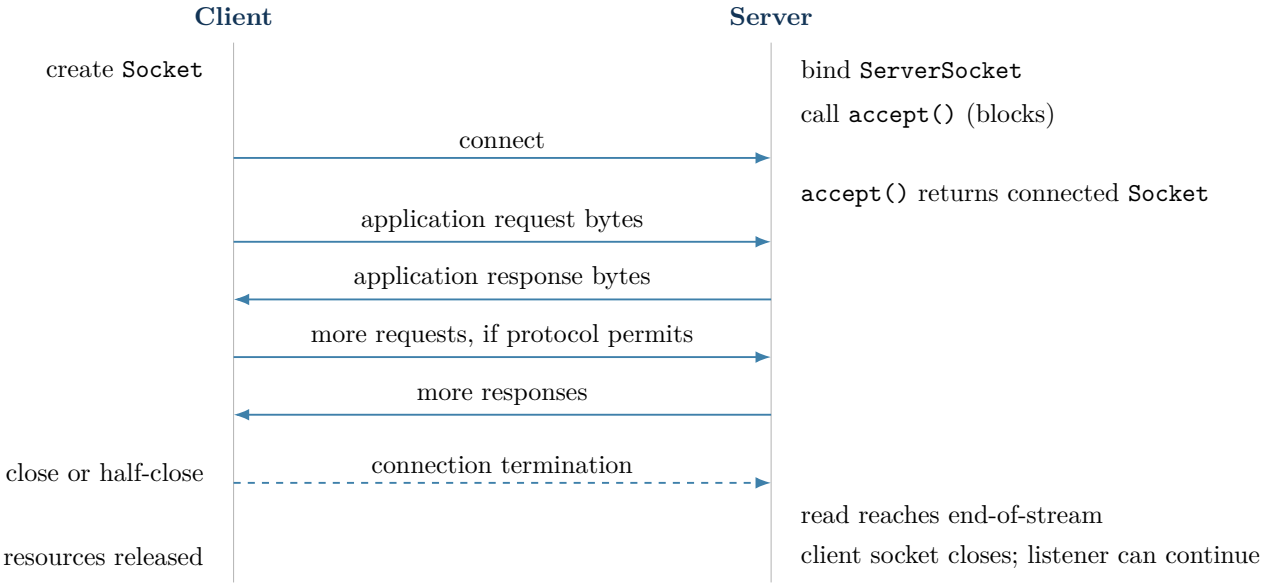


Figure 11: Conceptual lifecycle of a TCP client/server exchange. The application protocol determines the request and response sequence.

10.2 Blocking operations

In Java's standard blocking socket API:

- `ServerSocket.accept()` waits until a connection is available;
- `InputStream.read()` waits until bytes, end-of-stream, an error, or a configured timeout occurs;

- `BufferedReader.readLine()` waits for a line terminator, end-of-stream, an error, or a socket read timeout;
- writes can also block if buffers fill and the receiver or network is not accepting data quickly enough.

Blocking is simple for a first client/server program, but one wait must not indefinitely delay other clients or clean shutdown.

11. TCP Is a Byte Stream: Messages Need Framing

Do not match each sender call to `println()` or `write()` with one receiver call to `read()`. TCP preserves byte order, not call boundaries: a read may return part of one write or data from several writes.

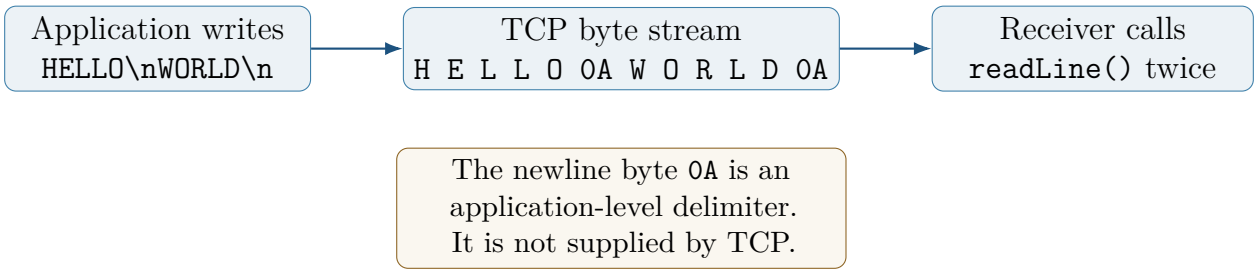


Figure 12: TCP carries bytes; the application-level newline delimiter reconstructs messages.

11.1 Common framing strategies

- **Delimiter-based:** terminate each message with a special byte sequence, such as newline.
- **Fixed length:** every message occupies the same number of bytes.
- **Length prefix:** send the payload length before the payload.
- **Self-describing structure:** parse a structured representation whose framing rules are well defined.
- **Connection close:** treat end-of-stream as the end of one response, useful only for specific protocol designs.

⚠ The classic newline deadlock

If the receiver calls `readLine()`, the sender must eventually send a line terminator. If the sender uses `print()` without a newline and does not close the connection, the receiver can wait forever. If both sides wait to read first, neither side sends anything and the conversation deadlocks.

One pair of writes can produce many valid read patterns

A sender may call `write("HELLO\n")` and then `write("WORLD\n")`. A receiver could read `HEL`, then `LO\nWORLD\n`; or it could receive both lines in one buffer. TCP guarantees byte order, not the boundaries of the sender's method calls. A delimiter-aware reader such as `readLine()` reconstructs the two application messages.

Framing strategy	Concrete example	Main caution
Delimiter-based	One command per newline in CALC/1.	The delimiter must be escaped, forbidden, or otherwise handled inside payload data.
Fixed length	A sensor record that is always exactly 32 bytes.	Wastes space for short values and becomes rigid when fields evolve.
Length prefix	Four-byte length followed by a binary or JSON payload.	The receiver must validate the length before allocating or waiting for data.
Self-describing structure	A format with explicit grammar, often inside an outer transport frame.	The parser still needs a reliable way to know where one top-level message ends.
Connection close	A protocol defines one response and uses end-of-stream as completion.	Prevents reuse and cannot represent several messages on one connection without extra rules.

11.2 Character encoding

Networks transport bytes, while Java `String` values contain characters, so both endpoints need an agreed encoding. The companion lab uses **UTF-8** through `StandardCharsets.UTF_8`; platform defaults may differ.

Encoding must be agreed, not guessed

The character `e` with an acute accent is represented by two bytes in UTF-8 but may be represented differently in another encoding. If the sender uses UTF-8 and the receiver assumes a platform-specific default, names and messages can be corrupted even though every byte arrived correctly. Both sides therefore specify `StandardCharsets.UTF_8` explicitly.

11.3 Serialization

Serialization converts structured values into a transmissible representation. In Week 1, a request such as `ADD 2 3` is a very small text serialization. Later, the course will use formats such as JSON, XML, and YAML.

12. Application Protocols: Giving Meaning to Bytes

Application protocol

An **application protocol** defines how communicating programs encode messages, what each message means, what order is legal, how completion is recognized, and how errors are represented.

A protocol normally specifies at least four dimensions:

1. **Syntax:** the structure of a valid message.
2. **Semantics:** the meaning of each command and field.
3. **Sequencing:** who sends first and which message may follow another.
4. **Failure behavior:** error responses, connection closure, timeouts, and retry rules.

12.1 A simple line-oriented calculator protocol

The lab uses a teaching protocol named `CALC/1`. Each message occupies one UTF-8 line.

CALC/1 protocol sketch

```
Server -> Client: READY CALC/1
Client -> Server: HELP
Server -> Client: OK COMMANDS ADD SUB MUL DIV QUIT

Client -> Server: ADD 2 3
Server -> Client: OK 5.0

Client -> Server: DIV 9 0
Server -> Client: ERR DIVISION_BY_ZERO

Client -> Server: QUIT
Server -> Client: BYE
```

The token before the remaining content acts as a status category:

- `OK` means the command succeeded;
- `ERR` means the server understood enough to report a protocol or application error;
- `READY` is a greeting and protocol-version declaration;
- `BYE` confirms orderly protocol termination.

12.2 Message grammar

A compact informal grammar is:

CALC/1 request grammar

```
request   := "HELP"
           | "QUIT"
           | operation SP number SP number

operation := "ADD" | "SUB" | "MUL" | "DIV"
number    := a decimal number accepted by Double.parseDouble
SP        := one or more whitespace characters
```

12.3 Validation rules

The server should never trust client text. It must:

- reject blank lines and unknown commands;
- verify the number of tokens;
- parse numeric values safely;
- reject division by zero;
- impose a maximum line length;
- return stable machine-readable error codes;
- decide whether the connection remains usable after an error.

12.4 Sequencing and protocol deadlock

A protocol is a conversation. Both sides must agree who sends first and when a response is complete.

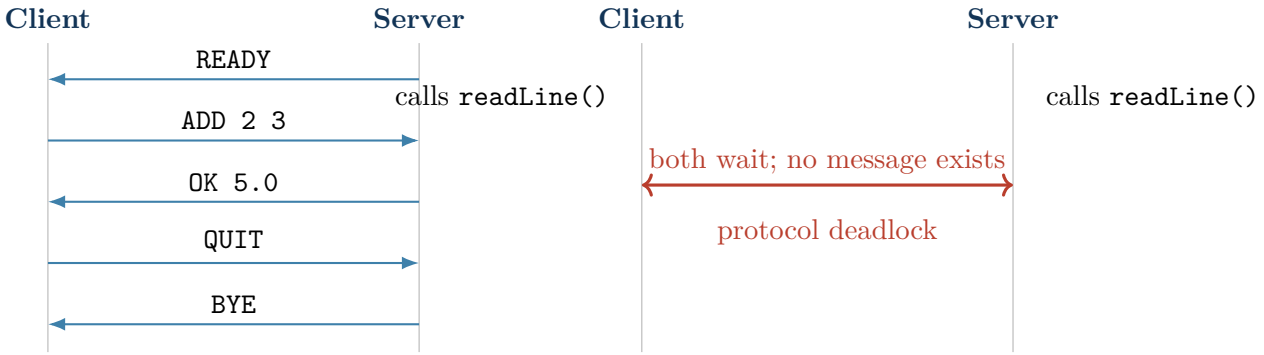
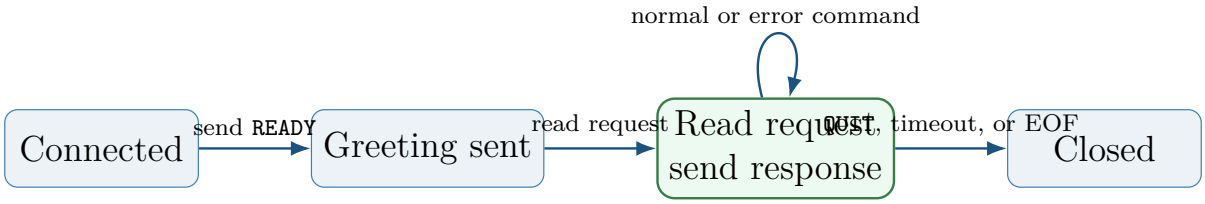


Figure 13: Left: a valid request-response sequence. Right: both sides read first and wait forever.

Documenting the conversation as a sequence diagram or state machine exposes these ordering assumptions before implementation.



Protocol state machine. Legal transitions make it explicit when each side reads, writes, repeats, or closes.

The state machine reveals missing responses, illegal transitions, and ambiguous closure rules, and it provides a direct structure for tests.

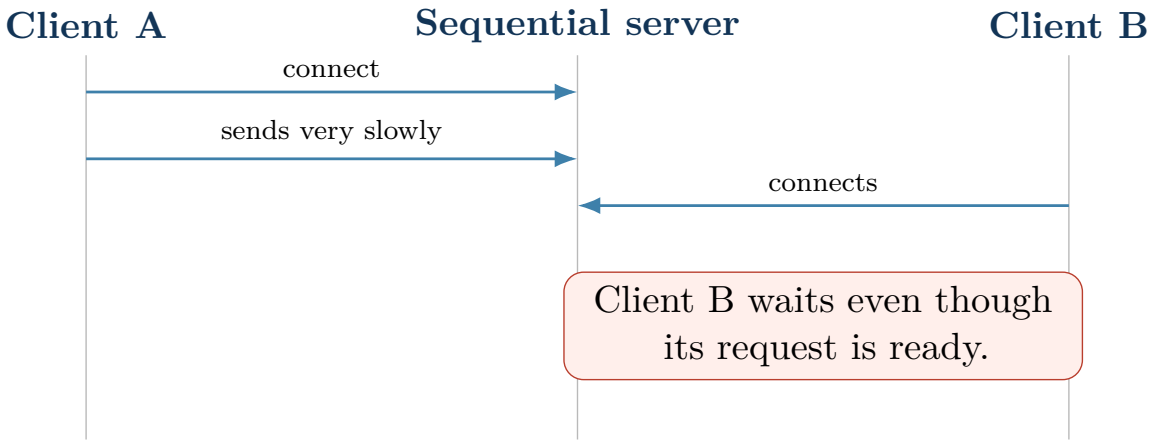
13. Sequential and Concurrent Servers

13.1 Sequential server

A sequential server accepts one client, handles it completely, closes its socket, and returns to `accept()`. The model is easy to reason about and suits the first lab.

```
accept → handle client A → close A → accept → handle client B
```

The limitation is **head-of-line blocking**: if client A sends slowly or remains connected, client B may wait even though the machine could perform more work.



Head-of-line blocking. One slow client can delay unrelated clients in a sequential server.

13.2 Concurrent server

A concurrent server separates connection acceptance from client handling, commonly with one listener and a fixed worker pool.

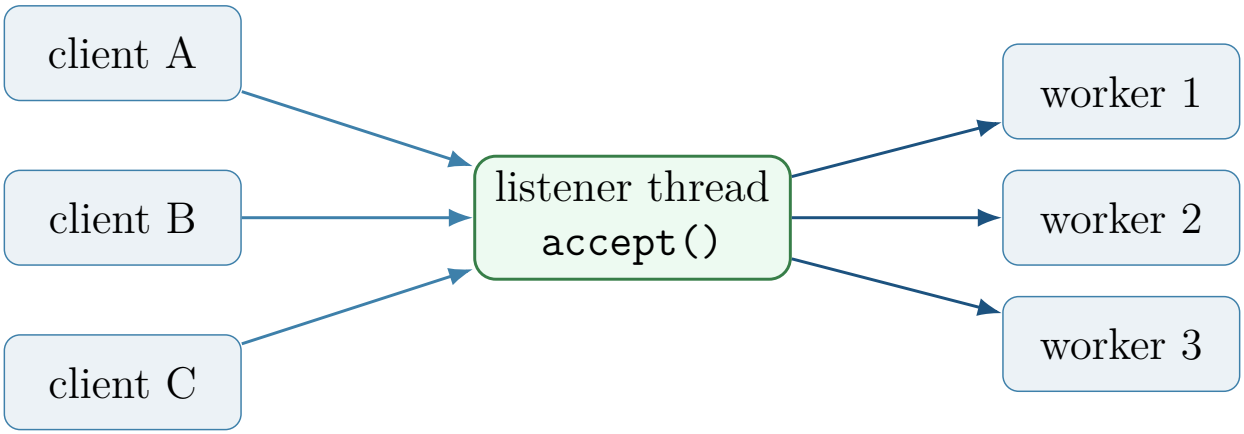


Figure 14: A listener accepts connections quickly and delegates client handling to workers.

Concurrency creates additional responsibilities:

- shared state must be synchronized or designed to avoid mutation;
- workloads need limits so a server is not overwhelmed;
- logs should identify each connection;
- one worker's failure must not terminate the listener;
- shutdown must close the listener and stop workers cleanly.

⚠ A fixed thread pool is not a complete production limit

`Executors.newFixedThreadPool(n)` limits the number of active worker threads, but its standard task queue is unbounded. A production server also needs admission control, bounded queues, request limits, timeouts, and overload behavior. The Week 1 version is a teaching model, not a deployment template.

14. Failures, Timeouts, and Defensive Design

14.1 Common failure symptoms

Symptom or exception	Likely meaning	Checks
`UnknownHostException`	The host name could not be resolved.	Check spelling, DNS, and whether a literal IP address works.
`ConnectException: Connection refused`	No listener accepted the connection at that address/port, or a firewall actively rejected it.	Start the server; verify address, port, and bind interface.

Symptom or exception	Likely meaning	Checks
`SocketTimeoutException`	A configured connect, accept, or read deadline expired.	Determine which operation timed out; inspect server load and protocol sequencing.
`SocketException: Connection reset`	The peer or network terminated the connection abruptly.	Check peer logs and whether one side crashed or closed unexpectedly.
`BindException: Address already in use`	Another socket is bound to the requested endpoint, or a prior use is still affecting rebinding.	Inspect listening ports; stop the other process or select another port.
`readLine()` returns `null`	End-of-stream: the peer closed its output side and no more characters remain.	Treat it as a normal disconnect unless the protocol required another message.
Program appears frozen	It may be blocked in `accept()`, `readLine()`, or a write.	Log before and after each blocking call; verify framing and send order.

14.2 Timeouts

A timeout converts an unbounded wait into a bounded one. Java's `Socket.setSoTimeout(ms)` limits a socket read, while the second argument to `Socket.connect(...)` sets a connection deadline. Expiry means only that the operation did not complete from the caller's perspective before that deadline; it does not prove that the remote effect never occurred. Retrying a state-changing operation can therefore duplicate effects unless the protocol supports safe retries.

Payment retry risk and idempotency

A payment service may charge a card and then lose its response. The portal times out and cannot tell whether the charge occurred. Blindly repeating the same operation can charge twice. A safer design associates the logical operation with a unique request or idempotency key so that a repeated request can return the existing outcome instead of applying the effect again.

14.3 Defensive protocol rules

- Bound input sizes and reject excessive requests.
- Validate every token; do not trust the client.
- Return stable error codes instead of stack traces.
- Log enough context to diagnose failures, but do not log secrets.
- Close sockets in all paths, preferably with try-with-resources.
- Decide whether a protocol error closes the connection or permits another request.

- Separate protocol parsing from network I/O so each can be tested independently.

15. Security Foundations for Raw Sockets

Week 1 does not yet implement authentication or TLS, but even a teaching socket program should follow basic safety rules:

- bind to `127.0.0.1` unless the instructor explicitly authorizes LAN testing;
- never treat client input as trusted;
- impose request-size, connection-count, and time limits;
- do not print internal stack traces or secrets to remote clients;
- close resources and handle unexpected disconnections;
- remember that encryption, authentication, and authorization are separate layers covered later.

⚠ Do not expose a teaching server to the public Internet

The examples are intentionally small and omit production controls such as TLS, authentication, authorization, rate limiting, audited configuration, bounded admission, and hardened parsing. Run them only in the authorized classroom or local environment.

16. Worked Design: From Requirement to Protocol

The earlier sections introduced each design decision separately; this worked example assembles them into one coherent contract.

Consider this requirement:

> A client must ask a remote service to perform one of four arithmetic operations. The server must validate input, return a clear result or error, support several requests per connection, and close cleanly when the client sends `QUIT`.

16.1 Step 1: Identify roles and endpoint

- Client: a command-line Java process.
- Server: a Java process listening on `127.0.0.1:5050`.
- Transport: TCP.
- Encoding: UTF-8.
- Framing: one newline-terminated message per request or response.

16.2 Step 2: Define requests and responses

Worked CALC/1 conversation

```
Server: READY CALC/1
Client: ADD 4 7
Server: OK 11.0
Client: DIV 9 0
Server: ERR DIVISION_BY_ZERO
Client: NOPE 1 2
Server: ERR UNKNOWN_COMMAND
Client: QUIT
Server: BYE
```

16.3 Step 3: Define sequencing

1. Server sends exactly one greeting after accepting the connection.
2. Client sends exactly one request line.
3. Server sends exactly one response line.
4. Steps 2–3 repeat until the client sends `QUIT`, a timeout occurs, or either endpoint closes.

16.4 Step 4: Anticipate failures

Condition	Planned behavior
Blank request	Return `ERR EMPTY_REQUEST`; keep the connection open.
Wrong token count	Return `ERR BAD_ARITY`; keep the connection open.
Invalid number	Return `ERR BAD_NUMBER`; keep the connection open.
Division by zero	Return `ERR DIVISION_BY_ZERO`; keep the connection open.
Client disconnects	`readLine()` returns `null`; log and close the socket.
Read timeout	Return no misleading result; log the timeout and close the connection.
Unexpected server exception	Log details locally; send only a generic error when safe, then close.

16.5 Step 5: Separate responsibilities

The implementation will be easier to test if it separates:

- network setup and socket lifecycle;
- reading and writing complete lines;

- parsing and validating the protocol;
- performing the arithmetic;
- formatting the response;
- logging and error handling.

17. Theory Consolidation

Key Points to Remember

- A distributed system coordinates autonomous elements through communication.
- Distribution enables sharing, scaling, availability, specialization, and geographic reach, but introduces latency, concurrency, security concerns, and partial failure.
- IPC includes shared-memory and message-passing mechanisms. Sockets are a message-passing foundation that can cross host boundaries.
- Real applications combine architectures and communication patterns such as client-server, multi-tier, work queues, publish-subscribe, streaming, load-balanced replicas, gateways, and fan-out/fan-in.
- An endpoint combines an address and port. A TCP connection joins two endpoints.
- TCP provides a reliable, ordered byte stream; it does not define application messages.
- Applications create framing, encoding, syntax, semantics, sequencing, validation, and error rules.
- `ServerSocket` listens and accepts; each accepted client is represented by a connected `Socket`.
- Blocking is simple but must be understood. Timeouts bound waits; concurrency lets several clients progress.
- Silence is ambiguous in a distributed system. Retries require careful semantics.

17.1 Self-check questions

1. Why can a process not normally read another process's ordinary Java variable?
2. What is the difference between a host, process, and thread?
3. Why is a port number needed in addition to an IP address?
4. What does `ServerSocket.accept()` return?
5. Why can `readLine()` block even after the sender called `print()`?
6. What application property does TCP preserve, and what message property does it not preserve?
7. Why can a timeout not prove that a remote operation never happened?

8. What is partial failure?
9. How does a concurrent server improve over a sequential server?
10. Which rules belong to the transport protocol, and which belong to the application protocol?
11. What is the difference between a distributed architecture and a communication pattern?
12. In the school-portal example, which components act as clients and servers, and why can those roles change?
13. When is a work queue preferable to direct request-response communication?
14. Why must an API gateway define behavior for partial downstream failure?
15. Choose TCP or UDP for course registration, a live game-position update, and a file upload. Explain the criterion used in each case.
16. Why can a unique request or idempotency key make retrying a state-changing operation safer?

A. Suggested Answers to Theory Self-Check

1. Each process has its own virtual address space and ordinary variables are not directly shared across that isolation boundary. IPC must explicitly share or transfer data.
2. A host is a computing environment; a process is a running program with isolated state; a thread is one execution sequence inside a process and shares that process's memory.
3. The IP address identifies the host/interface, while the port distinguishes transport endpoints and applications on that host.
4. It returns a connected `Socket` for one client; the original `ServerSocket` remains the listener.
5. `readLine()` waits for a newline or end-of-stream. `print()` without a newline does not complete the line, and flushing only transmits existing bytes.
6. TCP preserves reliable, in-order bytes within a live connection; it does not preserve application message boundaries.
7. The remote server may complete after the caller's deadline or the response may be lost. A timeout describes observation, not definitive remote state.
8. Partial failure occurs when one process or communication path fails while other system elements continue.
9. A concurrent server delegates accepted clients to workers so one slow client does not necessarily prevent others from progressing.

10. TCP defines the transport byte stream and connection behavior. The application protocol defines framing, encoding, commands, meanings, sequence, validation, and error responses.

11. Architecture allocates responsibilities and state among components; a communication pattern describes how those components interact, such as request-response, queueing, publish-subscribe, or streaming.

12. The browser is a client of the portal; the portal is a server to the browser but a client of authentication and student-record services; those services can in turn be clients of databases or other dependencies. Roles are contextual to each interaction.

13. A work queue is preferable when the sender does not require immediate final completion, work may be buffered, demand is bursty, or workers should scale and retry independently.

14. A gateway depends on several services. It must decide whether to fail the entire request, return a partial result, use cached data, translate errors, or retry when one dependency is slow or unavailable.

15. Course registration normally uses TCP because correctness and ordered confirmation matter; a live game-position update may use UDP because a newer update can replace a lost old one; a file upload uses TCP because every byte must arrive in order.

16. A unique key lets the receiver recognize that a retry represents the same logical operation and return the existing result rather than applying the state change a second time.

B. Glossary

Term	Concise definition
Address space	Memory-address environment visible to a process.
Bind	Associate a local socket with an address and port.
Blocking	Waiting in an operation until progress, completion, failure, closure, or timeout.
Client	Role that initiates an interaction.
Connection	TCP state joining two endpoints in a bidirectional byte stream.
Datagram	Self-contained message unit used by UDP.
Endpoint	Communication address, commonly an IP address and port.
Framing	Rules for locating application-message boundaries.
IPC	Mechanisms that let independent processes communicate and coordinate.
Latency	Time between initiating an operation and observing progress or completion.
Loopback	Local network path that returns traffic to the same host.
Partial failure	Failure of one component or path while others continue.

Term	Concise definition
Port	16-bit transport identifier used to distinguish endpoints on a host.
Process	Running program with independent operating-system state and virtual memory.
Protocol	Agreed syntax, semantics, sequencing, and failure rules.
Serialization	Conversion of structured values into a transmissible representation.
Server	Role that listens for and handles requests.
Socket	Programming abstraction for a communication endpoint.
Thread	Sequence of execution within a process.
Timeout	Configured maximum wait for an operation.
API gateway	Entry component that routes, protects, translates, or aggregates calls to internal services.
Broker	Intermediary that stores, routes, or distributes messages or events between producers and consumers.
Event	Record that a meaningful state change occurred, such as `RegistrationCreated`.
Fan-out/fan-in	Pattern that splits work among several workers and then combines their results.
Idempotency key	Client-supplied identifier used to recognize retries of the same logical operation and avoid duplicate effects.
Load balancer	Component that distributes requests or connections among several service instances.
Message queue	Buffered channel from which consumers retrieve work, often with retry and acknowledgement policies.
Publish-subscribe	Pattern in which producers publish by topic and multiple interested consumers receive events.
Replication	Maintaining several copies or instances to improve scale, availability, or geographic reach.
Request-response	Interaction in which one request is matched with one response or error.
Streaming	Long-lived transfer of a sequence of values or media chunks over time.
Tier	Architectural layer with a distinct responsibility, such as presentation, application logic, or data.

References and Further Reading

Maarten van Steen and Andrew S. Tanenbaum, *Distributed Systems*, 4th edition.

The official site provides the book's teaching resources, complete PDF/PPTX slides, figures, and code. The introductory material supports the definitions, architectures, communication concepts, and failure model used here.

https://www.distributed-systems.net/index.php/books/ds4/

James Kurose and Keith Ross, *Computer Networking: A Top-Down Approach*, 9th edition resources.

Official PowerPoint slides, videos, knowledge checks, interactive material, and Wireshark labs. The application and transport chapters support the process, socket, TCP, and UDP concepts.

https://gaia.cs.umass.edu/kurose_ross/ppt.php

Leonard Richardson and Sam Ruby, *RESTful Web Services*.

The author's official page provides a complete legal online edition. Chapter 1 helps connect raw distributed communication to programmable Web services.

https://www.crummy.com/writing/RESTful-Web-Services/

Oracle Java Tutorial: All About Sockets.

Introductory explanations and client/server examples, including the distinction between a listening server socket and the connected socket returned by `accept()`.

https://docs.oracle.com/javase/tutorial/networking/sockets/

Java SE 25 API: `java.net.Socket`.

Current official API contract for connected sockets, streams, endpoint information, options, and timeouts.

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/net/Socket.html

Java SE 25 API: `java.net.ServerSocket`.

Current official API contract for binding, listening, accepting, timeouts, and closing server sockets.

https://docs.oracle.com/en/java/javase/25/docs/api/java.base/java/net/ServerSocket.html

Java SE 25 API: `java.util.concurrent.Executors`.

Official documentation for fixed thread pools, virtual-thread executors, and executor behavior.

Official Java SE 25 `Executors` API documentation

Java Tutorial: The try-with-resources Statement.

Official explanation of deterministic resource closing for objects that implement `AutoCloseable`.

https://docs.oracle.com/javase/tutorial/essential/exceptions/tryResourceClose.html

RFC 9293: Transmission Control Protocol.

Current Internet Standards Track TCP specification. Section 2.2 summarizes the reliable, in-order byte-stream service and port-based multiplexing used in these notes.

https://www.rfc-editor.org/rfc/rfc9293.html

RFC 768: User Datagram Protocol.

The foundational UDP specification.

https://www.rfc-editor.org/rfc/rfc768.html

IANA Service Name and Transport Protocol Port Number Registry.

Official port registry and definitions of system, user, and dynamic/private ranges.

https://www.iana.org/assignments/service-names-port-numbers/

Microsoft Azure Architecture Center: Cloud Design Patterns.

A practical catalogue of commonly used distributed-system patterns, including gateway aggregation, publisher-subscriber, queue-based load leveling, circuit breaker, retry, and competing consumers.

https://learn.microsoft.com/en-us/azure/architecture/patterns/

Microsoft Azure Architecture Center: Gateway Aggregation Pattern.

A concrete description of a gateway that combines calls to several backend services and the latency, coupling, and failure trade-offs it introduces.

https://learn.microsoft.com/en-us/azure/architecture/patterns/gateway-aggregation